南方科技大学

LAB 4: Recurrent Neural Networks

COE201, Spring 2026

2026-03-16

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

Sequence Modeling Motivation 1

The Problem with Feedforward Networks 2

Understanding RNN 4

RNN Architecture 5

Implementation in PyTorch 10

Data Representation 11

Using `nn.RNN` 12

Data Prep: Why Padding? 15

Lab Tasks 16

Dataset Intro: SST-2 17

Lab 4 Exercises 18

Sequence Modeling Motivation

The Problem with Feedforward Networks

Problem: Standard feed-forward neural networks treat each input independently. But many real-world tasks involve **sequential data**:

- Text: "I love this movie"
- Time series data: Stock prices, sensor data

Example1 (text): "The movie was not good" vs "The movie was good, not bad"

- Same words, different meaning - **order matters**!

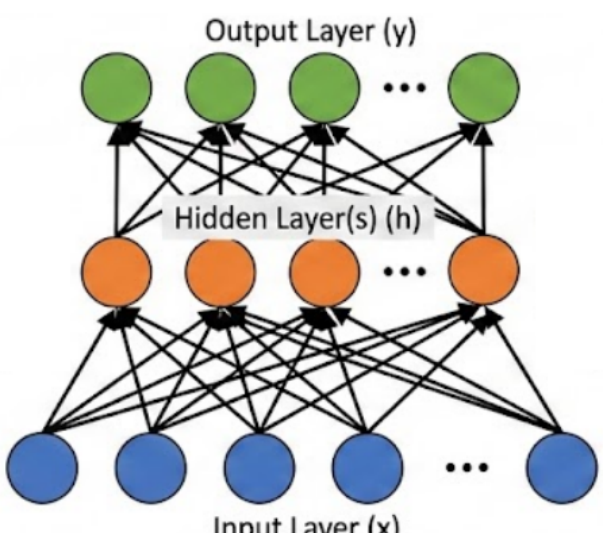
Example2 (stock price):

The Problem with Feedforward Networks

Solution: A network that can **remember** previous inputs (**need memory!**).

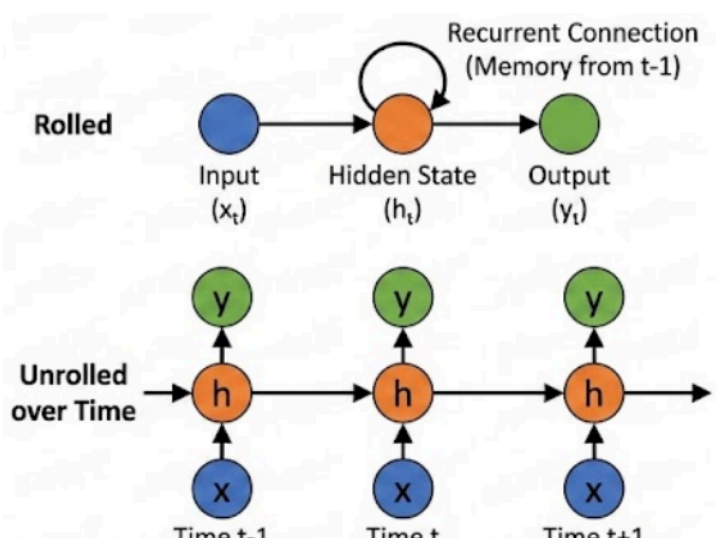
Feedforward NN:

- Each input processed independently
- No memory of past inputs



Recurrent NN:

- Maintains a **hidden state** carries information from previous steps



Understanding RNN

RNN Architecture

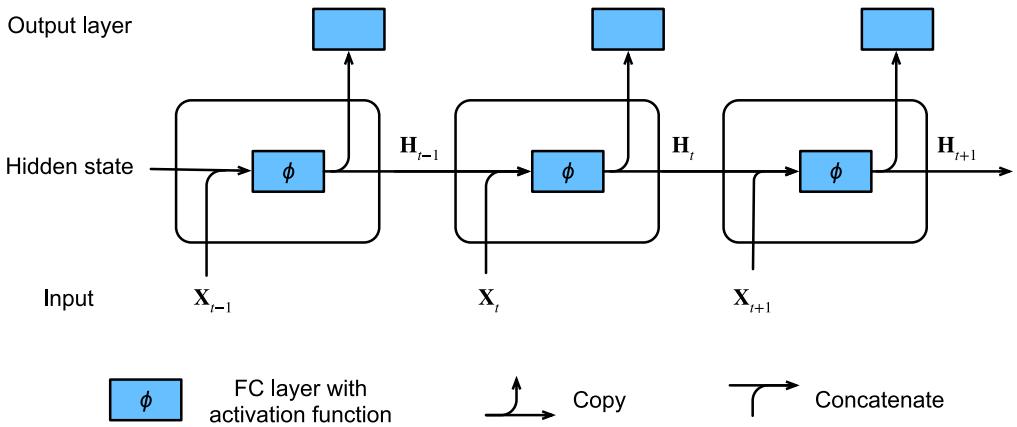
The RNN Cell

An RNN processes sequences one step at a time, maintaining a **hidden state**:

$$H_t = \tanh(W_h H_{t-1} + W_x x_t + b)$$

Where:

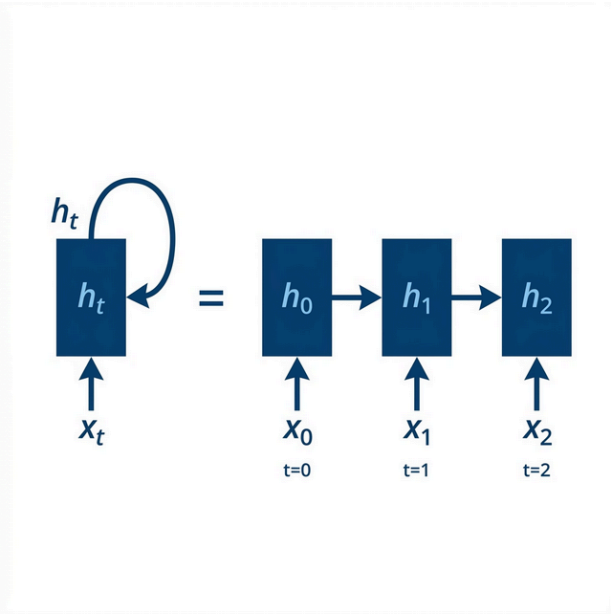
- x_t: input at time step t
- H_{t-1}: hidden state from previous step
- H_t: new hidden state (output)
- W_h, W_x, b: learnable parameters



RNN Architecture

Time Unrolling

An RNN unrolled in time is just a deep network with **shared weights**:



Key insight: The same parameters are used at every time step!

RNN Architecture

The Hidden State as "Summary"

Think of the **hidden state** H_t as a "running summary" of the sequence.

The Mental Model:

- H_0: "I've seen nothing yet"
- H_1: "I've seen 'The'"
- H_2: "I've seen 'The cat'"
- H_t: "I've seen everything from step 0 to t"

```python
# Conceptual update
memory = 0
for word in sentence:
    memory = combine(memory, word)

# The final 'memory' holds
# the context of the
# entire sentence!
```

Information Accumulation: At each step, we **blend** the new input (x_t) with our previous memory ($H_{\{t-1\}}$).

RNN Architecture

The Problem with Standard RNNs

The Example: `"I grew up in France... [100 words] ... I speak fluent [?]"`

To predict **"French"**, the model must remember **"France"** from 100 steps ago.

1. Vanishing Gradients

- Gradients multiplied by W_h every step.
- If W_h is small, the signal "fades".
- **Intuition**: The model **forgets** "France" by the time it reaches "French".

2. Exploding Gradients

- If W_h is large, gradients grow exponentially.
- **Result**: Parameters explode (`NaN`).
- **Solution**: **Gradient Clipping**.

RNN Architecture

Beyond Vanilla RNN: LSTM & GRU

Standard RNNs struggle with long-term memory due to vanishing gradients.

Modern Gated RNNs:

- **LSTM** (Long Short-Term Memory)
- **GRU** (Gated Recurrent Unit)
- Use "Gates" to protect memory, allowing gradients to flow over hundreds of steps.

PyTorch Implementation:

One-line drop-in replacement!

```python
# Just replace RNN with LSTM
self.rnn = nn.LSTM(embed_dim, hidden_dim,
                   batch_first=True)
```

While LSTM/GRU improved things, they are still **sequential** (slow $O(N)$ for-loop).

- **This Course**: We won't dwell on the complex internal gates of LSTM/GRU.
- **Next Step**: We will soon introduce **Transformers**.

Implementation in PyTorch

Data Representation

Sequence Tensor Shapes

Critical tensor shapes when processing sequences in PyTorch:

Data Component	Shape	Meaning
Input Data	`(Batch, Seq, Feat)`	Batch of sequences
Hidden State	`(Batch, Hidden)`	Memory at a specific time step
Model Output	`(Batch, Seq, Vocab)`	Output distribution at each step

```python
# Example: Batch of 16 text snippets, each 25 characters long.
# We embed each character into a 32-dim vector.
x_embedded = torch.randn(16, 25, 32)  # (Batch, Seq, Feat)
```

Using nn.RNN

1. Initialization (__init__)

```python
# input_size:  Dimension of each element in the sequence (e.g., embed_dim)
# hidden_size: Dimension of the memory/hidden state
# batch_first: If True, input shape is (Batch, Seq, Feature)
rnn = nn.RNN(input_size=10, hidden_size=20, batch_first=True)
```

2. The Forward Pass

```python
# x shape: (Batch, Seq_Len, Input_Size)
# out shape: (Batch, Seq_Len, Hidden_Size) -> Hidden state for EVERY step
# hn  shape: (Num_Layers, Batch, Hidden_Size) -> Hidden state for LAST step only
out, hn = rnn(x)
```

Tensor	What it represents
out	The “history” of hidden states across all time steps.
hn	The final “summary” used for prediction/classification.

Under the Hood: The Recurrence Loop

How does nn.RNN work internally? It's just a **for-loop** over the sequence!

```python
# Conceptual PyTorch Implementation
def forward(x, h_0):
    # x: (Batch, Seq_Len, Dim)
    outputs = []
    h_t = h_0
    for t in range(seq_len):
        x_t = x[:, t, :] # Current step
        h_t = torch.tanh(W_ih @ x_t + b_ih + W_hh @ h_t + b_hh)
        outputs.append(h_t)

    return torch.stack(outputs, dim=1), h_t
```

Using nn.RNN

Building a Sentiment Classifier

In classification, we typically "squeeze" the sequence into a single vector using the **last** hidden state.

```python
class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn       = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc        = nn.Linear(hidden_dim, 2)
    def forward(self, x):
        # 1. Embed words: (B, S) -> (B, S, E)
        x = self.embedding(x)
        # 2. Process sequence: Return (out, hn)
        _, hn = self.rnn(x)
        # 3. Predict using LAST hidden state: (1, B, H) -> (B, H)
        return self.fc(hn.squeeze(0))
```

Data Prep: Why Padding?

Sentences have different lengths, but PyTorch requires **rectangular tensors** for batching.

```python
import torch
from torch.nn.utils.rnn import pad_sequence

sequences = [torch.tensor([1, 2, 3]), torch.tensor([4, 5])]
# Pad to max length (3)
padded = pad_sequence(sequences, batch_first=True, padding_value=0)
# Result: [[1, 2, 3], [4, 5, 0]]
```

Row	Sequence
1	"The movie was great"
2	"Terrible movie! <PAD> <PAD>"

Lab Tasks

Complete the exercises in `lab4/`.

Dataset Intro: SST-2

The **Stanford Sentiment Treebank** (SST-2) is the industry-standard benchmark for **Binary Sentiment Classification**.

- Sentiment Analysis (detecting if text is happy/sad, positive/negative).
- **Task Type**: Classification (input text and output wether it is positive or negative).
- **Source**: Movie reviews from Rotten Tomatoes.
- **Labels**: 0 = Negative, 1 = Positive.
- **Size**: 67,000 sentences.

Sentence	Label
`"a visually stunning masterpiece"`	1
`"a complete waste of time"`	0

Lab 4 Exercises

1. **Task 1: Warm-up - Sequence Shapes (40 points)**
   - Complete the verification in `lab4_warmup.py`.
   - Understand the relationship between `output` and `hn`.
2. **Task 2: RNN Sentiment Classifier (SST-2 Dataset) (60 points)**
   - Complete the `SentimentRNN` class in `lab4_rnn.py`.
   - Implement both the layer definition and the `forward` pass.
   - **[Thought Question]** Understand the padding implementation in `collate_batch` and why **Pre-padding** is used here.
   - Skeleton code is provided in `lab4_rnn.py`.

Guide: Implementing GRU (Bonus)

GRU is a variant of **RNN** that uses **gates** to control the flow of information.

Refer to: https://zh.d2l.ai/chapter_recurrent-modern/gru.html

The mathematical equations for `CustomGRUCell`:

1. **Gates**: $r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$ (Reset Gate)
   $z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$ (Update Gate)
2. **Candidate & State Update**: $\tilde{h}_t = \tanh(W \cdot [r_t \cdot h_{t-1}, x_t] + b)$ (Candidate)
   $h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$ (Final)

Lab 4 Exercises

1. **Task 3 [Bonus]: GRU**
   - Complete the `CustomGRUCell` and `SentimentGRU` in `lab4_gru.py`.
   - Evaluate your GRU on **Long** (>35 words) vs. **Short** reviews within the test set.
   - Observe how well your GRU handles different sequence lengths!

Thanks for Listening!